



FUNDAMENTAL OF EMBEDDED SYSTEM FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA

SOIL HUMIDITY MONITOR

GROUP 15

Izzan Nawa Syarif	2306266956
R. Aisha Syauqi Ramadhani	2306250554
Fauzan Farras Hakim Budi H.	2306250610
Matthew Immanuel Sitorus	2306221024

PREFACE

Segala puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan proyek akhir Praktikum Sistem Embedded ini dengan lancar dan sesuai jadwal. Laporan ini dibuat sebagai salah satu persyaratan dalam menyelesaikan praktikum yang telah kami ikuti selama satu semester.

Proyek akhir yang kami kerjakan mengangkat topik Soil Humidity Monitor menggunakan sensor DHT11, dilengkapi dengan mekanisme interrupt dan timer untuk pengendalian pompa air secara otomatis, serta sistem notifikasi menggunakan LED dan buzzer. Praktikum ini memberikan kesempatan bagi kami untuk mengaplikasikan teori yang telah dipelajari di bangku kuliah ke dalam praktik langsung, meningkatkan kemampuan analisis, serta mengembangkan keterampilan teknis di bidang sistem embedded.

Dalam proses penyusunan laporan ini, kami memperoleh banyak bantuan dan dukungan dari berbagai pihak, baik secara langsung maupun tidak langsung. Oleh karena itu, kami mengucapkan terima kasih yang sebesar-besarnya kepada asisten laboratorium serta rekan-rekan satu tim atas kerja sama, dedikasi, dan kontribusinya yang telah membantu kelancaran proyek ini.

Kami menyadari bahwa laporan ini masih jauh dari sempurna dan terdapat kekurangan yang perlu diperbaiki. Dengan rendah hati, kami menerima kritik dan saran yang membangun demi peningkatan kualitas laporan di masa yang akan datang.

Akhir kata, kami berharap laporan ini dapat memberikan manfaat bagi para pembaca dan turut berkontribusi positif dalam pengembangan ilmu pengetahuan khususnya di bidang sistem embedded. Semoga Tuhan Yang Maha Esa selalu melimpahkan rahmat dan petunjuk-Nya kepada kita semua.

Depok, May 2025

Group 15

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION.....	4
1.1 PROBLEM STATEMENT.....	4
1.2 PROPOSED SOLUTION.....	4
1.3 ACCEPTANCE CRITERIA.....	5
1.4 ROLES AND RESPONSIBILITIES.....	6
1.5 TIMELINE AND MILESTONES.....	6
CHAPTER 2: IMPLEMENTATION4.....	7
2.1 HARDWARE DESIGN AND SCHEMATIC.....	7
2.2 SOFTWARE DEVELOPMENT.....	9
2.3 HARDWARE AND SOFTWARE INTEGRATION.....	21
CHAPTER 3: TESTING AND EVALUATION	23
3.1 TESTING.....	23
3.2 RESULT.....	23
3.3 EVALUATION.....	24
CHAPTER 4: CONCLUSION.....	25

CHAPTER 1

INTRODUCTION

1.1 PROBLEM STATEMENT

Kelembapan tanah merupakan salah satu faktor penting dalam keberhasilan pertumbuhan tanaman. Dalam praktiknya, pemantauan dan pengaturan kelembapan tanah masih banyak dilakukan secara manual oleh petani atau pemilik lahan, yang tentu memerlukan waktu, tenaga, dan keterlibatan secara langsung. Ketika terjadi kelalaian dalam penyiraman atau tidak adanya pemantauan secara berkala, kondisi tanah dapat menjadi terlalu kering atau terlalu basah. Hal ini dapat berdampak buruk terhadap pertumbuhan dan kesehatan tanaman, serta menurunkan hasil panen secara keseluruhan.

Di sisi lain, kemajuan teknologi sistem embedded dan sensor digital membuka peluang untuk merancang sistem monitoring kelembapan tanah secara otomatis yang dapat bekerja secara real-time dan mandiri. Namun, implementasi sistem seperti ini masih memiliki tantangan, seperti keterbatasan dalam konsumsi daya, respons terhadap perubahan kondisi lingkungan, serta integrasi antara komponen sensor, aktuator, dan mikrokontroler. Oleh karena itu, dibutuhkan suatu sistem monitoring berbasis sensor yang dilengkapi dengan fitur interrupt dan timer, serta dilengkapi dengan sistem notifikasi, agar dapat menjalankan kontrol terhadap pompa air secara otomatis berdasarkan kondisi kelembapan tanah yang terukur.

1.2 PROPOSED SOLUTION

Dalam proyek ini, sistem dirancang untuk mendeteksi tingkat kelembapan tanah secara otomatis dan real-time menggunakan sensor kelembapan digital (DHT11) yang terhubung dengan mikrokontroler slave. Data kelembapan dikirimkan ke mikrokontroler master melalui komunikasi SPI (Serial Peripheral Interface). Berdasarkan nilai kelembapan yang diterima, sistem akan menentukan apakah pompa air perlu diaktifkan. Berikut merupakan rincian solusi dari tiap komponen yang digunakan:

- Sensor DHT11: Sensor ini digunakan untuk mengukur kelembapan secara real-time. Data kelembapan yang diperoleh akan menjadi dasar pengambilan keputusan sistem, apakah pompa air perlu dinyalakan atau tidak.

- Mikrokontroler Master: Sistem ini menggunakan dua mikrokontroler, master dan slave. Slave bertugas membaca data dari sensor DHT11, lalu mengirimkannya ke master melalui komunikasi SPI. Master kemudian mengolah data dan menjalankan logika kontrol.
- Komunikasi SPI: SPI digunakan sebagai jalur komunikasi data digital antara master dan slave. Protokol ini dipilih karena memiliki kecepatan tinggi dan sederhana dalam implementasi, sehingga cocok untuk pengiriman data sensor secara cepat dan akurat.
- LED Indikator: LED digunakan sebagai indikator status kelembapan tanah:
 - LED merah menyala jika kelembapan $<$ ambang batas (tanah kering).
 - LED hijau menyala jika kelembapan $\geq$ ambang batas (tanah cukup lembap).
- Interrupt: Interrupt eksternal digunakan untuk mengatur waktu penyiraman secara manual.
 - INT0 berfungsi untuk menambah durasi penyiraman.
 - INT1 berfungsi untuk mengurangi durasi penyiraman.
- Timer1 Overflow: Timer1 digunakan untuk menghitung mundur durasi penyiraman. Saat overflow, counter dikurangi satu. Ketika counter > 0 , sistem akan tetap dalam mode penyiraman (indikator hijau menyala). Jika counter = 0, sistem kembali ke mode default berdasarkan sensor.
- Pompa Air: Pompa air dikendalikan secara otomatis berdasarkan data kelembapan dan waktu penyiraman. Pompa aktif (menyiram) saat tanah kering dan timer aktif, serta nonaktif saat tanah cukup lembap atau timer habis. Dalam simulasi ini, pompa dapat direpresentasikan melalui LED hijau sebagai indikator penyiraman aktif.

1.3 ACCEPTANCE CRITERIA

Soil Humidity Monitor dapat dikatakan berhasil berjalan apabila memenuhi syarat-syarat berikut:

- Sensor mampu membaca nilai kelembapan tanah secara real-time dan menghasilkan data yang stabil serta konsisten.
- Sistem berhasil mengirim dan menerima data kelembapan tanpa kesalahan atau keterlambatan.

- LED indikator dan pompa bekerja sesuai logika sistem berdasarkan nilai kelembapan dan status penyiraman.
- Tombol interrupt merespons dengan baik, dan sistem mampu menghitung mundur waktu penyiraman secara tepat menggunakan timer.

1.4 ROLES AND RESPONSIBILITIES

The roles and responsibilities assigned to the group members are as follows:

Roles	Responsibilities	Person
Role 1	Debugging, SPI, Sensor Interfacing	Izzan Nawa Syarif
Role 2	Laporan, Interrupt, Readme, Timer	R. Aisha Syauqi R.
Role 3	SPI, I/O, Hardware, Interrupt, Github, Proteus, Debugging	Fauzan Farras Hakim
Role 4	UART, PPT, Chart	Matthew Immanuel S.

Table 1. Roles and Responsibilities

1.5 TIMELINE AND MILESTONES

No	Agenda	Mei																	
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
1	Perumusan ide																		
2	Finalisasi Ide																		
3	Pengembangan Software																		
4	Penyelesaian Design Hardware																		
5	Integrasi dan Ujicoba Hardware dan Software																		
6	Assembly Final Product																		
7	Project Final Report																		
8	Deadline Pengumpulan																		

Fig 1. Gantt Chart

CHAPTER 2

IMPLEMENTATION

2.1 HARDWARE DESIGN AND SCHEMATIC

Untuk memenuhi kriteria pada proyek akhir Praktikum Sistem Embedded, maka pemrograman *software* dilakukan dalam bahasa Assembly untuk merancang program *humidity* ini. Sebelum menuliskan kode dalam bahasa Assembly, dibutuhkan implementasi rangkaian secara virtual. Dalam hal ini, kami menggunakan *software* Proteus untuk merangkai *humidity* berdasarkan *flowchart* yang sudah kami buat.

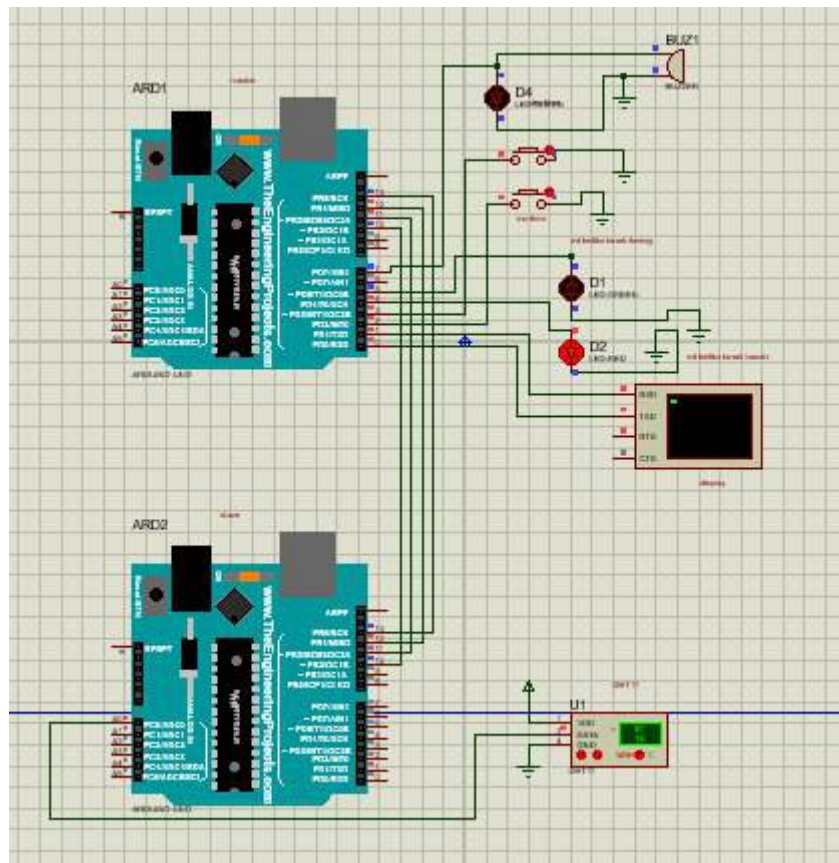


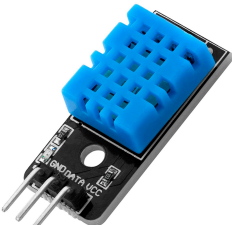
Fig 2. Proteus Design

Gambar di atas merupakan implementasi rangkaian secara virtual berdasarkan *flowchart* yang telah kami buat. Pada visualisasi ini, kelompok kami menggunakan beberapa komponen utama, yaitu sensor kelembapan tanah digital DHT11, LED indikator, dan beberapa tombol sebagai kontrol manual penyiraman dengan durasi berbeda. Penggunaan tombol tersebut merupakan implementasi dari interrupt

eksternal sesuai dengan pembelajaran pada Modul 7 (Interrupt). Selain itu, terdapat komunikasi data menggunakan protokol SPI untuk pengiriman data kelembapan dari sensor ke mikrokontroler utama, yang merupakan implementasi dari Modul 8 & 9 (I2C, SPI, dan Sensor Interfacing). Data kelembapan yang diterima akan ditampilkan secara real-time menggunakan serial monitor sebagai output, sesuai dengan pembelajaran dari Modul 4 (Serial Port).

Timer digunakan sebagai penghitung mundur durasi penyiraman yang dapat diatur secara manual melalui tombol yang memanfaatkan konsep timer dan interrupt dari Modul 7 dan penghitungan aritmatika untuk logika kontrol pompa air sesuai dengan Modul 5 (Arithmetic). Seluruh perangkat lunak dalam sistem ini diprogram menggunakan bahasa Assembly, sesuai dengan materi pada Modul 2 (Introduction to Assembly & I/O Programming), sehingga seluruh komponen dapat saling berinteraksi secara otomatis dan real-time dalam monitoring kelembapan tanah dan pengendalian pompa air.

Alat dan bahan:

Hardware			Software
Jumlah	Barang	Gambar	
2	Arduino		Arduino Uno
1	DHT11		Proteus

2	Button		Proteus
1	Buzzer		Proteus
3	LED		Proteus
Kabel Jumper			

2.2 SOFTWARE DEVELOPMENT

Untuk memenuhi kriteria proyek akhir Praktikum Sistem Embedded, maka pemrograman *software* dilakukan dalam bahasa Assembly untuk merancang program *humidity* ini. Bahasa Assembly adalah sebuah bahasa pemrograman rendah (low-level) yang digunakan untuk memprogram komputer, khususnya mikroprosesor dan mikrokontroler. Bahasa ini memiliki keterkaitan yang erat dengan *machine learning* yang merupakan bahasa yang dipahami langsung oleh komputer. Sebelum memulai pembuatan kode bahasa Assembly, *flowchart* harus dibuat sebagai acuan pembuatan kode.

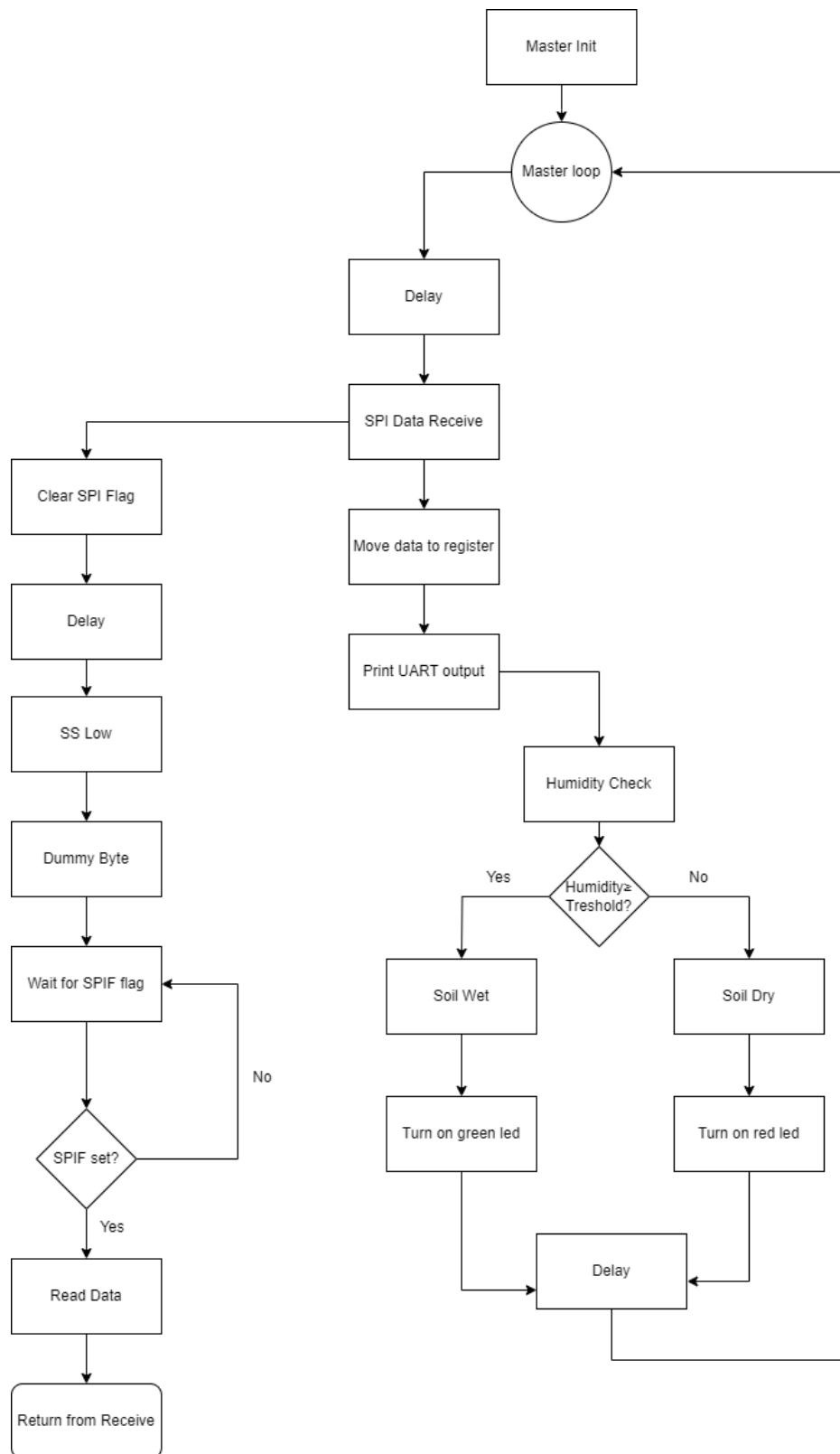


Fig 3.1 Master Flowchart

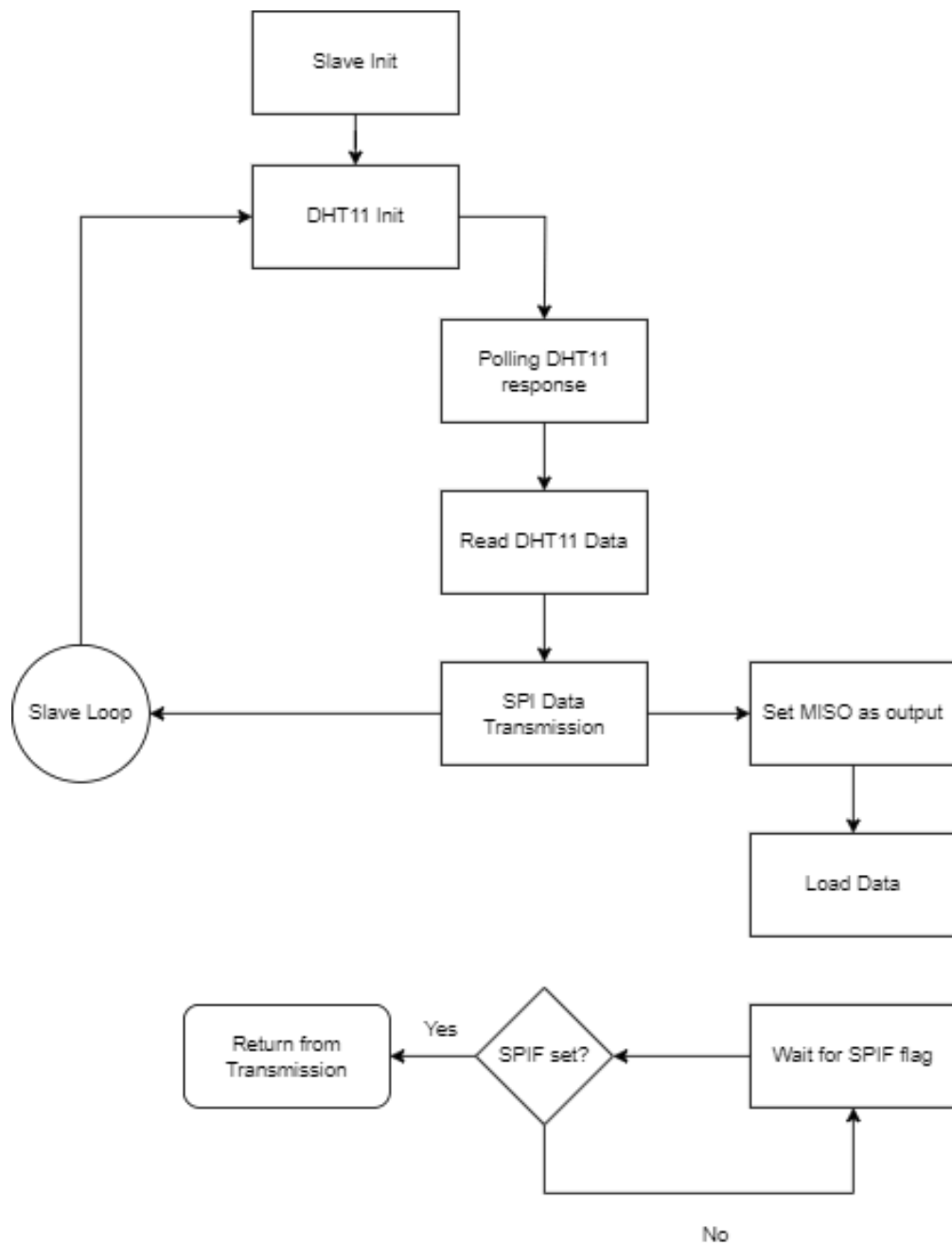


Fig 3.2 *Slave Flowchart*

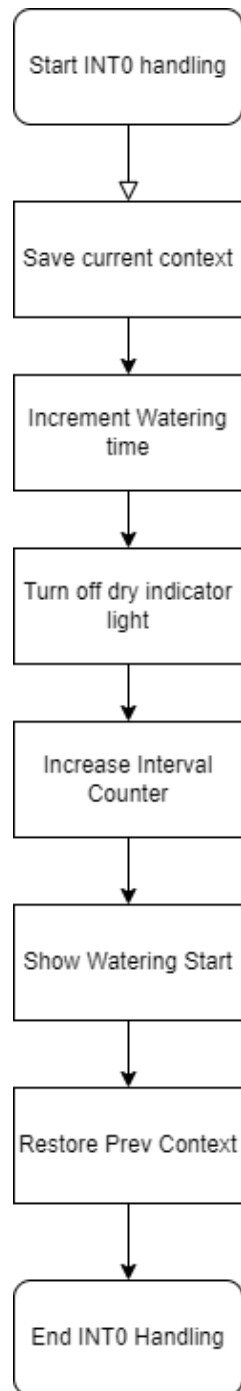


Fig 3.3 INT0 handling *Flowchart*

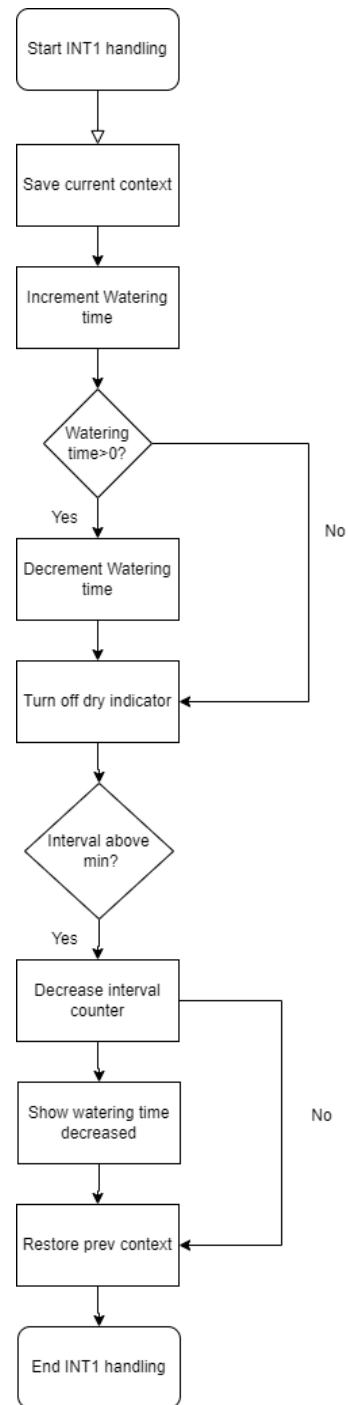


Fig 3.4 INT1 handling *Flowchart*

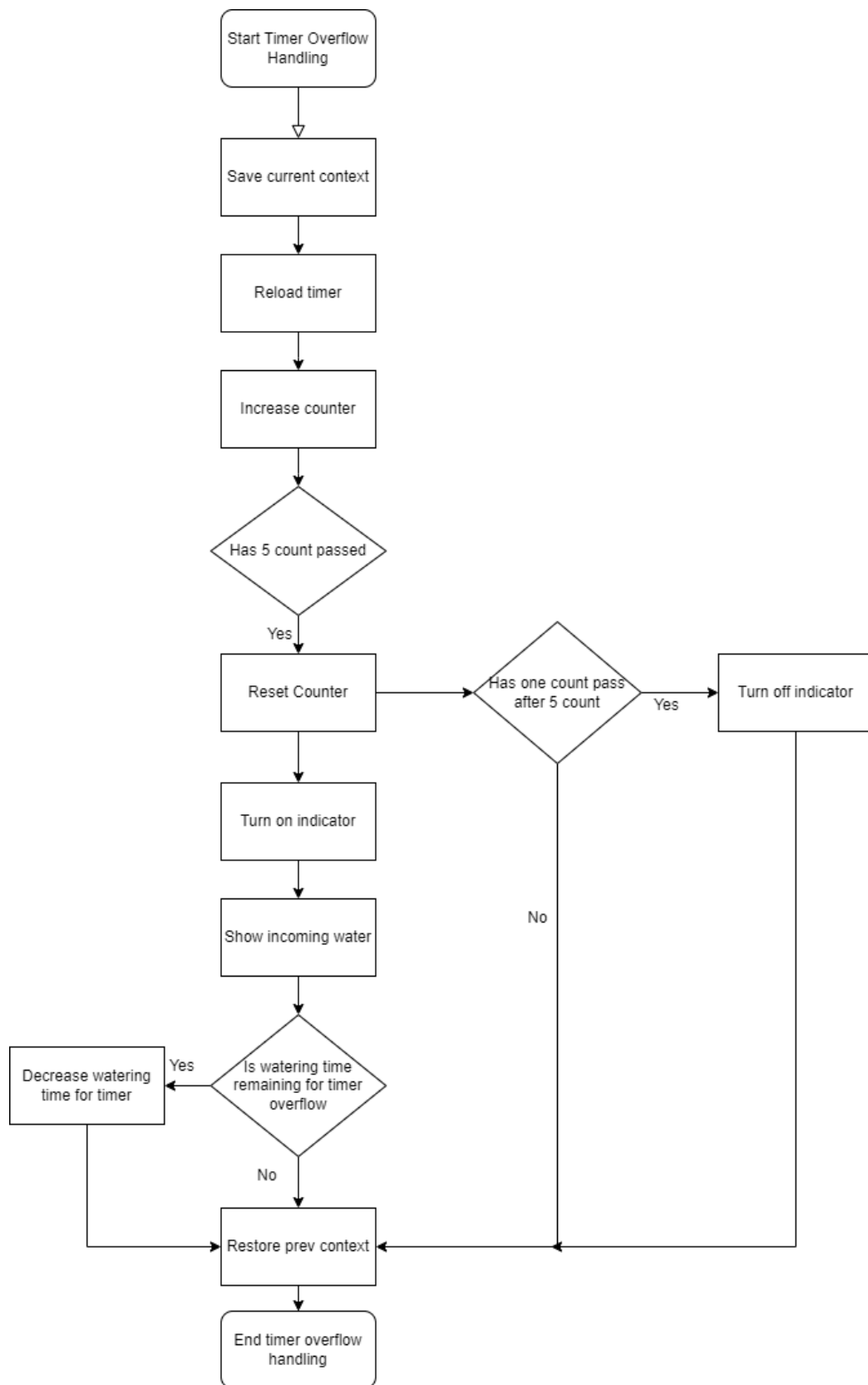


Fig 3.5 *Timer1 Flowchart*

Gambar diatas merupakan alur kerja program yang divisualisasikan dengan *flowchart* dengan *software* Draw.io. Alur *humidity* ini bekerja diawali dengan inisialisasi seluruh perangkat, seperti sensor kelembapan, komunikasi SPI, dan timer. Setelah itu, sistem akan membaca data kelembapan tanah dari sensor secara berkala. Data yang diperoleh kemudian diproses untuk menentukan status kelembapan tanah. Jika nilai kelembapan berada di bawah ambang batas yang telah ditentukan, sistem akan mengaktifkan pompa air untuk melakukan penyiraman tanah. Sebaliknya, jika kelembapan tanah sudah mencukupi, pompa akan dimatikan agar tidak terjadi penyiraman berlebih. Selain itu, sistem juga menampilkan indikator LED yang menunjukkan kondisi kelembapan secara visual. Selama proses berjalan, interrupt akan memonitor tombol untuk menyesuaikan durasi penyiraman secara manual. Timer akan menghitung durasi penyiraman dan secara otomatis mematikan pompa setelah waktu habis. Program akan terus melakukan siklus ini secara berulang untuk memastikan kelembapan tanah tetap terjaga secara optimal.

Menurut pada *flowchart* yang telah dibuat, rangkaian *humidity* dibuat dengan menggunakan *software* arduino IDE dengan menggunakan bahasa Assembly. Berikut kode lengkapnya:

Master	
<pre> #define __SFR_OFFSET 0x00 #include "avr/io.h" .global main .global INT0_vect .global INT1_vect .global TIMER1_OVF_vect ; Register assignments: ; R16 - Temporary register for operations ; R17 - Humidity value from SPI ; R18 - Timer counter ; R19 - Second counter for 5-second timing ; R20-R22 - Used for delays ; Constants .equ HUMIDITY_THRESHOLD, 50 ; Threshold for wet/dry soil (50%) main: ; Initialize stack pointer (SP) LDI R16, hi8(RAMEND) OUT SPH, R16 LDI R16, lo8(RAMEND) OUT SPL, R16 ; Configure PORTD outputs SBI DDRD, 4 ; Set PD4 as output (Green LED - Wet indicator) SBI DDRD, 5 ; Set PD5 as </pre>	<pre> ; Convert binary value (0-255) to decimal digits MOV R16, R30 CLR R24 ; UART transmit function uart_transmit: PUSH R17 uart_wait: LDS R17, UCSR0A SBRS R17, UDRE0 ; Wait until data register is empty RJMP uart_wait STS UDR0, R16 ; Send character POP R17 RET 12: LDS R17, UCSR0A SBRS R17, UDRE0 ;test data buffer if data can be sent RJMP 12 STS UDR0, R16 ;print ADC mid digit on serial monitor POP R16 ;restore ADCH value from STACK register ANDI R16, 0x0F ;mask & extract low-nibble ADD R16, R23 MOV R28, R16 </pre>

<pre> output (Red LED/Buzzer - Dry indicator) SBI DDRD, 7 ; Set PD7 as output (Additional timer LED) ; Initialize SPI as master SBI DDRB, 5 ; Set PB5 as output (SCK) SBI DDRB, 3 ; Set PB3 as output (MOSI) SBI DDRB, 2 ; Set PB2 as output (SS) ;SBI PORTB, 2 ; Set SS high SBI PORTD, 2 ; Enable pull-up on PD2 (INT0) SBI PORTD, 3 ; Enable pull-up on PD3 (INT1) ; Enable INT0 and INT1, falling edge LDI R16, (1<<INT0) (1<<INT1) OUT EIMSK, R16 LDI R16, (1<<ISC01) (1<<ISC11) STS EICRA, R16 LDI R16, hi8(49911) ; Load high byte first STS TCNT1H, R16 LDI R16, lo8(49911) ; Then load low byte STS TCNT1L, R16 ; 1024 prescaler and overflow interrupt LDI R16, (1<<CS12) (1<<CS10) STS TCCR1B, R16 LDI R16, (1<<TOIE1) STS TIMSK1, R16 init_serial: CLR R24 STS UCSR0A, R24 STS UBRROH, R24 LDI R24, 103 STS UBRR0L, R24 ;to set baud rate 9600 LDI R24, 1<<RXEN0 1<<TXEN0 STS UCSR0B, R24 ;enable RXB & TXB LDI R24, 1<<UCSZ00 1<<UCSZ01 STS UCSROC, R24 ; SPI: Enable, Master mode, Clock Rate fck/16 LDI R16, (1<<SPE) (1<<MSTR) (1<<SPR0) OUT SPCR, R16 ; LEDs start off CBI PORTD, 4 CBI PORTD, 5 CBI PORTD, 7 ; Clear initial values CLR R17 CLR R18 CLR R19 LDI R25, 5 SEI ; Enable global interrupts run: ; Main program loop RCALL delay_500ms </pre>	<pre> SUBI R28, 58 BRPL A_F_LSD 13: LDS R17, UCSR0A SBRS R17, UDRE0 ;test data buffer if data can be sent RJMP 13 STS UDR0, R16 ;print ADC LSD on serial monitor 14: LDS R17, UCSR0A SBRS R17, UDRE0 ;test data buffer if data can be sent RJMP 14 LDI R18, 0x0A STS UDR0, R18 ;print newline on serial monitor 15: LDS R17, UCSR0A SBRS R17, UDRE0 ;test data buffer if data can be sent RJMP 15 STS UDR0, R18 ;print carriage return on serial monitor RCALL delay_sec ;1 second delay RET ;===== ===== A_F_MSD: ADD R16, R24 ;add 7 to byte to get ASCII chars A to F RJMP 12 A_F_LSD: ADD R16, R24 ;add 7 to byte to get ASCII chars A to F RJMP 13 delay_sec: ;1s delay LDI R20, 255 16: LDI R21, 255 17: LDI R22, 80 18: DEC R22 BRNE 18 DEC R21 BRNE 17 DEC R20 BRNE 16 RET ;----- ----- ; == INT0: Tambah Waktu == INT0_vect: PUSH R16 IN R16, SREG PUSH R16 INC R18 CBI PORTD, 5 ; Hijau OFF INC R25 RCALL print_timer_add POP R16 OUT SREG, R16 POP R16 RETI </pre>
---	--

<pre> RCALL delay_500ms RCALL delay_500ms RCALL SPI_RECEIVE ; Receive humidity data via SPI MOV R16, R17 MOV R29, R17 RCALL print_ADC RCALL CHECK_HUMIDITY ; Check humidity and set LEDs RCALL delay_500ms RJMP run ; Loop forever ; SPI receive gets humidity value from slave SPI_RECEIVE: ; First clear any pending flags IN R16, SPSR IN R16, SPDR RCALL delay_100ms CBI PORTB, 2 LDI R16, 0x00 OUT SPDR, R16 SPI_WAIT: IN R16, SPSR ; Check SPI status SBRS R16, SPIF ; Skip if SPIF flag is set RJMP SPI_WAIT ; Wait for transmission to complete ;SBI PORTB, 2 ; Set SS high to end transmission IN R17, SPDR ; Read received humidity data R17 ;ROL R17 RET ; Check humidity value and set appropriate LED CHECK_HUMIDITY: ;turn off all LEDs to avoid any overlap CBI PORTD, 4 CBI PORTD, 5 ; compare with threshold and set appropriate LED CPI R29, HUMIDITY_THRESHOLD ; Compare humidity with threshold BRSH SOIL_WET ; Branch if humidity >= threshold (soil is wet) ; Soil is dry - turn on only red LED/buzzer SBI PORTD, 5 ; Turn on red LED/buzzer RET SOIL_WET: ; Soil is wet - turn on only green LED SBI PORTD, 4 ; Turn on green LED RET ; Delay functions delay_100ms: LDI R20, 100 ; Outer counter outer1: LDI R21, 200 ; Middle counter </pre>	<pre> ; === INT1: Kurangi Waktu === INT1_vect: PUSH R16 IN R16, SREG PUSH R16 TST R18 BREQ no_dec DEC R18 no_dec: CBI PORTD, 5 CPI R25, 6 BRLO min_reached DEC R25 RCALL print_timer_dec min_reached: POP R16 OUT SREG, R16 POP R16 RETI print_timer_add: PUSH R16 ; Print "Timer add 1" LDI R16, 'T' RCALL uart_transmit LDI R16, 'i' RCALL uart_transmit LDI R16, 'm' RCALL uart_transmit LDI R16, 'e' RCALL uart_transmit LDI R16, 'r' RCALL uart_transmit LDI R16, ' ' RCALL uart_transmit LDI R16, 'a' RCALL uart_transmit LDI R16, 'd' RCALL uart_transmit LDI R16, 'd' RCALL uart_transmit LDI R16, ' ' RCALL uart_transmit LDI R16, '1' RCALL uart_transmit ; Print current timer value LDI R16, ' ' RCALL uart_transmit LDI R16, '(' RCALL uart_transmit ; Convert R25 to ASCII and print MOV R16, R25 SUBI R16, -48 ; Convert to ASCII (add 48) RCALL uart_transmit LDI R16, 's' RCALL uart_transmit LDI R16, ')' RCALL uart_transmit ; Print newline and carriage return LDI R16, 0x0A ; Newline RCALL uart_transmit LDI R16, 0x0D ; Carriage return </pre>
---	--

<pre> middle1: LDI R22, 200 ; Inner counter inner1: DEC R22 BRNE inner1 DEC R21 BRNE middle1 DEC R20 BRNE outer1 RET delay_500ms: LDI R20, 250 ; Longer delay for testing outer2: LDI R21, 200 middle2: LDI R22, 200 inner2: DEC R22 BRNE inner2 DEC R21 BRNE middle2 DEC R20 BRNE outer2 RET print_ADC: MOV R30, R29 ; Store original value in R30 ; Print "Temperature: " string LDI R16, 'T' RCALL uart_transmit LDI R16, 'e' RCALL uart_transmit LDI R16, 'm' RCALL uart_transmit LDI R16, 'p' RCALL uart_transmit LDI R16, 'e' RCALL uart_transmit LDI R16, 'r' RCALL uart_transmit LDI R16, 'a' RCALL uart_transmit LDI R16, 't' RCALL uart_transmit LDI R16, 'u' RCALL uart_transmit LDI R16, 'r' RCALL uart_transmit LDI R16, 'e' RCALL uart_transmit LDI R16, ':' RCALL uart_transmit LDI R16, ' ' RCALL uart_transmit MOV R16, R30 CLR R24 CLR R23 ; Calculate hundreds digit hundreds_loop: CPI R16, 100 BRLO tens ; If less than 100, move to tens SUBI R16, 100 INC R24 ; Increment hundreds counter RJMP hundreds_loop </pre>	<pre> RCALL uart_transmit POP R16 RET print_timer_dec: PUSH R16 ; Print "Timer dec 1" LDI R16, 'T' RCALL uart_transmit LDI R16, 'i' RCALL uart_transmit LDI R16, 'm' RCALL uart_transmit LDI R16, 'e' RCALL uart_transmit LDI R16, 'r' RCALL uart_transmit LDI R16, ' ' RCALL uart_transmit LDI R16, 'd' RCALL uart_transmit LDI R16, 'e' RCALL uart_transmit LDI R16, 'c' RCALL uart_transmit LDI R16, ' ' RCALL uart_transmit LDI R16, '1' RCALL uart_transmit ; Print current timer value LDI R16, ' ' RCALL uart_transmit LDI R16, '(' RCALL uart_transmit ; Convert R25 to ASCII and print MOV R16, R25 SUBI R16, -48 ; Convert to ASCII (add 48) RCALL uart_transmit LDI R16, 's' RCALL uart_transmit LDI R16, ')' RCALL uart_transmit ; Print newline and carriage return LDI R16, 0x0A ; Newline RCALL uart_transmit LDI R16, 0x0D ; Carriage return RCALL uart_transmit POP R16 RET ; == Timer1 Overflow: 5-Second Timer for PD7 == TIMER1_OVF_vect: PUSH R16 IN R16, SREG PUSH R16 ; Reset Timer1 for next 1-second interval LDI R16, hi8(49911) ; Load high byte first (for 1-second timing) STS TCNT1H, R16 LDI R16, lo8(49911) ; Then load low byte STS TCNT1L, R16 ; This interrupt fires </pre>
--	---

```

        ; Calculate tens digit
tens:
    CPI    R16, 10
    BRLO   ones           ; If less
than 10, move to ones
    SUBI   R16, 10
    INC    R23             ; Increment
tens counter
    RJMP   tens

ones:

    ; Print hundreds digit (only if
non-zero)
    TST    R24
    BREQ   skip_hundreds  ; Skip
printing if hundreds = 0
    MOV    R16, R24
    LDI    R24, 48
    ADD    R16, R24        ; Convert to
ASCII
    RCALL  uart_transmit

skip_hundreds:

    TST    R24
    BRNE   print_tens     ; If
hundreds was printed, always print tens
    TST    R23
    BREQ   skip_tens      ; Skip
printing if tens = 0

print_tens:
    MOV    R16, R23
    LDI    R23, 48
    ADD    R16, R23        ; Convert to
ASCII
    RCALL  uart_transmit

skip_tens:
    MOV    R16, R30        ; Get
original value

    ; Get ones digit
    MOV    R16, R30        ; Get
original value
    CLR    R24             ; Clear
hundreds
    CLR    R23             ; Clear tens

hundreds_loop2:
    CPI    R16, 100
    BRLO   tens2          ; If less
than 100, move to tens
    SUBI   R16, 100
    INC    R24             ; Increment
hundreds counter
    RJMP   hundreds_loop2

tens2:
    CPI    R16, 10
    BRLO   ones2          ; If less
than 10, move to ones
    SUBI   R16, 10
    INC    R23             ; Increment
tens counter
    RJMP   tens2

ones2:
    ; R16 now has ones digit, convert
to ASCII
    LDI    R24, 48
    ADD    R16, R24        ; Convert to
ASCII

```

```

approximately every second
    INC    R19             ; Increment
second counter
    CP     R19, R25        ; Check if
we've reached 5 seconds
    BRNE   not_five_sec

    ; Five seconds have passed
    CLR    R19             ; Reset
second counter

    ; Toggle PD7 LED - turn ON
    SBI    PORTD, 7        ; Turn ON PD7
LED

    RCALL  print_water_message

    ; Decrement timer counter if active
    TST    R18
    BREQ   skip_dec        ; If counter
= 0, don't decrement
    DEC    R18             ; Decrement
timer counter
skip_dec:
    RJMP   timer_exit

not_five_sec:
    ; For other seconds, turn OFF PD7
    CPI    R19, 1         ; Check if
we're at 1 second after the 5-second
mark
    BRNE   timer_exit
    CBI    PORTD, 7        ; Turn OFF
PD7 LED after 1 second

timer_exit:
    POP    R16
    OUT    SREG, R16
    POP    R16
    RETI

print_water_message:
    PUSH   R16

    LDI    R16, 'W'
    RCALL  uart_transmit
    LDI    R16, 'A'
    RCALL  uart_transmit
    LDI    R16, 'T'
    RCALL  uart_transmit
    LDI    R16, 'E'
    RCALL  uart_transmit
    LDI    R16, 'R'
    RCALL  uart_transmit
    LDI    R16, ' '
    RCALL  uart_transmit
    LDI    R16, 'I'
    RCALL  uart_transmit
    LDI    R16, 'N'
    RCALL  uart_transmit
    LDI    R16, 'C'
    RCALL  uart_transmit
    LDI    R16, 'O'
    RCALL  uart_transmit
    LDI    R16, 'M'
    RCALL  uart_transmit
    LDI    R16, 'I'
    RCALL  uart_transmit
    LDI    R16, 'N'
    RCALL  uart_transmit
    LDI    R16, 'G'
    RCALL  uart_transmit

    LDI    R16, 0x0A        ; Newline
    RCALL  uart_transmit

```

<pre> RCALL uart_transmit ; Print newline and carriage return LDI R16, 0x0A RCALL uart_transmit LDI R16, 0x0D RCALL uart_transmit RET </pre>	<pre> LDI R16, 0x0D ; Carriage return RCALL uart_transmit POP R16 RET </pre>
Slave	
<pre> ;Data Transmission menggunakan SPI ;Register used: R20, R26, R27 ;===== transmitted: SBI DDRB, 4 ;Set MISO for slave transmission LDI R20, (1<<SPE) ;Enable SPI sebagai Slave OUT SPCR, R20 ;Transmission OUT SPDR, R26 ;Transmit data pada R26 done: in R27, SPSR ;Cek interrupt flag SPI SBRS R27, SPIF RJMP done ;Trasfer belum selesai kembali ke label "done" ret ;Debugging subroutine ;===== print_ADC: ;init usart init_serial: CLR R24 STS UCSR0A, R24 ;clear UCSR0A register STS UBRR0H, R24 ;clear UBRR0H register LDI R24, 103 ;& store in UBRR0L 103 STS UBRR0L, R24 ;to set baud rate 9600 LDI R24, 1<<RXEN0 1<<TXEN0 STS UCSR0B, R24 ;enable RXB & TXB LDI R24, 1<<UCSZ00 1<<UCSZ01 STS UCSR0C, R24 ;asynch, no parity, 1 stop, 8 bits LDI R23, 48 ;constants used to get ASCII values LDI R24, 7 ;for chars 0-->9 & A-->F ;ADD R25, R23 ;add 48 to byte to get ASCII char 0 to 9 ;----- PUSH R16 ;store copy of ADCH in STACK register ANDI R16, 0xF0 ;mask & extract high-nibble SWAP R16 ;swap high-nibble with low-nibble ADD R16, R23 ;add 48 to byte to get ASCII char 0 to 9 MOV R28, R16 ;store a copy of byte in R28 SUBI R28, 58 ;subtract 58 from R28 BRPL A_F_MSD ;jump if result is +ve lo2: LDS R17, UCSR0A SBRS R17, UDRE0 ;test data buffer if data can be sent RJMP 12 ;----- STS UDR0, R16 ;print ADC mid digit on serial monitor ;----- POP R16 ;restore ADCH value from STACK register ANDI R16, 0x0F ;mask & extract low-nibble ADD R16, R23 MOV R28, R16 SUBI R28, 58 BRPL A_F_LSD </pre>	

```

;-----
lo3: LDS    R17, UCSR0A
      SBRS   R17, UDRE0      ;test data buffer if data can be sent
      RJMP   lo3
;-----
      STS    UDR0, R16      ;print ADC LSD on serial monitor
;-----
lo4: LDS    R17, UCSR0A
      SBRS   R17, UDRE0      ;test data buffer if data can be sent
      RJMP   lo4
;-----
      LDI    R18, 0x0A
      STS    UDR0, R18      ;print newline on serial monitor
;-----
lo5: LDS    R17, UCSR0A
      SBRS   R17, UDRE0      ;test data buffer if data can be sent
      RJMP   lo5
;-----
      LDI    R18, 0x0D
      STS    UDR0, R18      ;print carriage return on serial monitor
;-----
      RCALL  delay_sec      ;1 second delay
      RET
;=====
A_F_MSD:
      ADD    R16, R24        ;add 7 to byte to get ASCII chars A to F
      RJMP   lo2
;-----
A_F_LSD:
      ADD    R16, R24        ;add 7 to byte to get ASCII chars A to F
      RJMP   lo3

delay_sec:                                ;1s delay
      LDI    R20, 255
lo6: LDI    R21, 255
lo7: LDI    R22, 80
lo8: DEC    R22
      BRNE   lo8
      DEC    R21
      BRNE   lo7
      DEC    R20
      BRNE   lo6
      RET

```

Berdasarkan kode, fungsi DHT11_sensor berfungsi untuk menginisialisasi dan memulai komunikasi dengan sensor DHT11 guna membaca nilai kelembapan tanah secara digital. Fungsi ini mengatur sinyal start dan response yang diperlukan untuk menerima data sensor, lalu menyimpan nilai kelembapan ke dalam register khusus. Fungsi DHT11_reading bertugas membaca data bit-bit kelembapan yang dikirimkan sensor secara berurutan, kemudian mengonversi bit-bit tersebut menjadi nilai kelembapan dalam format byte yang dapat diproses lebih lanjut.

Fungsi SPI_send_data digunakan untuk mengirimkan data kelembapan dari mikrokontroler pengukur ke mikrokontroler pengendali melalui protokol SPI. Fungsi ini mengatur jalur komunikasi dan memastikan data terkirim secara sinkron dan akurat. Selain itu, fungsi LED_control bertanggung jawab mengatur status LED indikator berdasarkan nilai kelembapan yang diterima: LED merah menyala jika

kelembapan berada di bawah batas ambang, menandakan tanah kering, sedangkan LED hijau menyala jika kelembapan sudah mencukupi.

Fungsi `interrupt_init` menginisialisasi dan mengaktifkan interrupt eksternal yang digunakan untuk mengatur durasi penyiraman secara manual melalui tombol. Fungsi `timer1_overflow` digunakan sebagai timer untuk menghitung mundur durasi penyiraman secara otomatis, di mana pompa air akan tetap aktif selama timer berjalan dan akan mati saat waktu habis. Fungsi `pump_control` mengatur kondisi pompa air berdasarkan nilai kelembapan dan timer, sehingga penyiraman hanya dilakukan saat tanah kering dan waktu penyiraman masih tersisa.

2.3 HARDWARE AND SOFTWARE INTEGRATION

Tidak mungkin suatu rangkaian dapat bekerja tanpa adanya kode yang diunggah ke dalam Arduino. Proses integrasi antara *software* dan *hardware* hadir untuk mengatasi permasalahan tersebut. Dalam proses integrasi ini, rangkaian harus disesuaikan antara input dan outputnya dengan kode yang sudah dibuat. Input sensor kelembapan DHT11 ditempatkan pada pin digital nomor 7 Arduino, sedangkan input tombol untuk mengatur penyiraman manual dan durasi timer ditempatkan pada beberapa pin digital lain, misalnya tombol START di pin 3 dan tombol STOP di pin 4. Semua tombol dipastikan memiliki resistor pull-up yang dihubungkan ke ground untuk menghindari noise.

Output dari rangkaian berupa LED indikator kelembapan dan pompa air yang dikendalikan menggunakan pin-pin digital Arduino lainnya. LED hijau akan menyala jika kelembapan berada di bawah ambang batas, sedangkan LED hijau menyala jika kelembapan sudah mencukupi. Pompa air juga diatur oleh Arduino untuk menyiram tanaman sesuai kondisi kelembapan dan durasi timer yang telah dipilih. Semua kontrol dan pembacaan sensor ini diatur dengan kode yang diunggah ke Arduino menggunakan bahasa Assembly, sehingga *hardware* dan *software* dapat bekerja secara sinkron.

Setelah pemasangan pin dan perangkat keras dilakukan, langkah berikutnya adalah pengujian kode yang sudah dibuat. Saat program dijalankan, sensor DHT11 akan membaca nilai kelembapan tanah secara digital. Namun, sensor baru akan mulai bekerja setelah tombol START ditekan. Sistem akan memonitor kelembapan dan

timer yang dipilih untuk menentukan berapa lama pompa akan menyiram tanaman. Nilai kelembapan dan status timer akan ditampilkan melalui LED indikator dan diproses oleh mikrokontroler untuk memastikan penyiraman berjalan sesuai kondisi yang diinginkan.

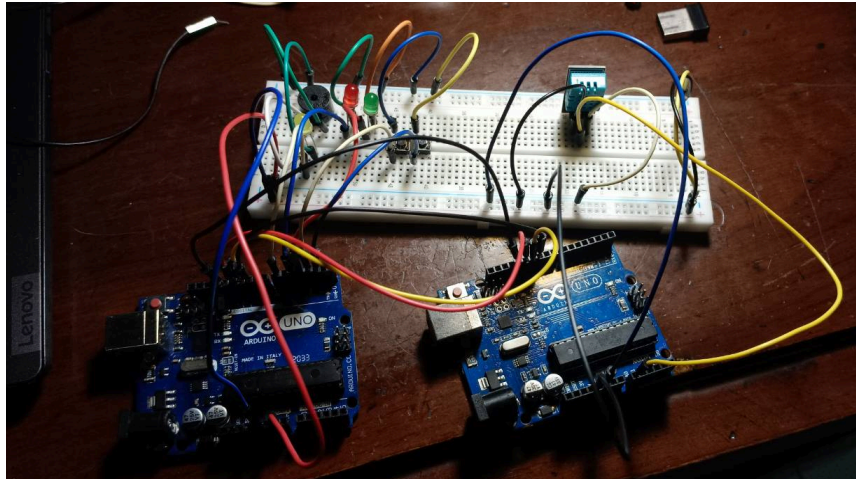


Fig 4. *Integration between Software and Hardware*

CHAPTER 3

TESTING AND ANALYSIS

3.1 TESTING & RESULT

Setelah melakukan integrasi antara *hardware* dan *software*, rangkaian diuji untuk membuktikan apakah sistem bekerja sesuai dengan input kode yang telah diunggah. Pengujian dilakukan dengan memberikan berbagai kondisi kelembapan tanah yang berbeda untuk memastikan sensor DHT11 dapat membaca data secara akurat dan mikrokontroler dapat memproses data tersebut dengan benar. Pada pengujian ini, tombol START ditekan untuk memulai pembacaan sensor dan sistem kemudian mengaktifkan pompa air sesuai dengan nilai kelembapan yang terdeteksi. LED indikator menyala sesuai dengan kondisi kelembapan, dimana LED merah menunjukkan tanah kering dan LED hijau menunjukkan tanah sudah cukup lembab. Timer penyiraman juga diuji dengan menekan tombol durasi yang berbeda untuk memastikan pompa menyala dan mati sesuai waktu yang dipilih. Hasil pengujian menunjukkan bahwa sistem dapat membaca nilai kelembapan secara akurat, mengendalikan pompa air sesuai kondisi tanah, dan menyesuaikan durasi penyiraman sesuai timer yang dipilih. Dengan demikian, rangkaian dan program yang dibuat sudah berjalan sesuai dengan tujuan awal proyek ini.

3.2 ANALYSIS

Setelah dilakukan pengujian, didapatkan hasil yang sesuai dengan harapan dan tujuan proyek. Sensor DHT11 mampu membaca nilai kelembapan tanah dengan akurat dan mengirimkan data secara tepat ke mikrokontroler. Respons sistem terhadap perubahan kelembapan juga berjalan dengan baik, dimana pompa air aktif secara otomatis saat kelembapan turun di bawah batas yang telah ditentukan.

Pengaturan timer untuk durasi penyiraman juga berjalan efektif, sesuai dengan pilihan waktu yang dipilih melalui tombol input. LED indikator berhasil menampilkan status kelembapan secara visual dengan benar, memudahkan pemantauan kondisi

tanah secara langsung. Hasil pengujian ini menunjukkan bahwa integrasi antara *hardware* dan *software* berjalan lancar dan stabil selama pengoperasian.

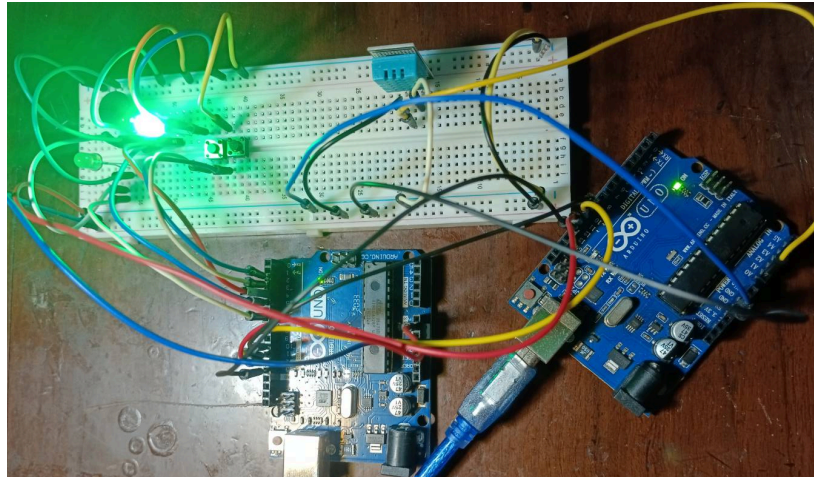


Fig 5. *Testing Result Part 1*

3.3 EVALUATION

Evaluasi pada proyek akhir sistem embedded ini dibagi menjadi dua aspek, yaitu evaluasi kinerja selama proses pengerjaan proyek dan evaluasi hasil yang diperoleh setelah implementasi. Dari segi kinerja selama pengerjaan proyek, kelompok kami menunjukkan kerja sama yang baik sehingga proses pengerjaan berjalan terorganisir dan tidak berantakan. Pembagian tugas dilakukan secara seimbang dan tepat waktu, sehingga semua kebutuhan proyek dapat diselesaikan sesuai dengan rencana yang telah ditetapkan. Sedangkan dari segi hasil yang didapatkan, kelompok kami berhasil mencapai tujuan dan harapan awal proyek. Seluruh modul yang dirancang telah berhasil diimplementasikan dalam kode assembly dengan baik, sehingga proyek ini dapat dikatakan memenuhi semua persyaratan dari proyek akhir sistem embedded. Sebagai langkah pengembangan berikutnya, kami menyarankan untuk mengintegrasikan sistem dengan aplikasi mobile atau IoT agar monitoring kelembapan tanah dan pengendalian pompa bisa dilakukan secara jarak jauh. Selain itu, peningkatan penggunaan sensor dengan akurasi lebih tinggi dan optimasi algoritma pengendalian pompa juga dapat dilakukan untuk meningkatkan efisiensi penggunaan air serta keandalan sistem secara keseluruhan.

CHAPTER 4

4.1 CONCLUSION

Berdasarkan hasil pengujian dan analisis yang telah dilakukan, dapat disimpulkan bahwa sistem pengukuran kelembapan tanah berbasis mikrokontroler dengan sensor DHT11 telah berhasil dirancang dan diimplementasikan dengan baik sesuai rancangan awal. Sistem ini mampu membaca data kelembapan tanah secara akurat dan konsisten, serta melakukan pengiriman data antar mikrokontroler melalui protokol komunikasi SPI secara efektif dan sinkron. Hal ini membuktikan bahwa mekanisme komunikasi data dalam sistem berjalan dengan lancar tanpa adanya gangguan signifikan.

Selain itu, kontrol pompa air yang terintegrasi dengan timer penyiraman otomatis juga berfungsi sesuai harapan. Pompa air dapat aktif secara otomatis saat nilai kelembapan tanah berada di bawah batas ambang yang telah ditentukan dan akan berhenti ketika durasi penyiraman selesai, sehingga sistem ini dapat mengoptimalkan penggunaan air dan menghindari pemborosan. Implementasi indikator LED sebagai penanda status kelembapan juga berjalan dengan baik, memberikan informasi visual yang mudah dipahami oleh pengguna mengenai kondisi tanah secara real-time.

Integrasi antara *hardware* dan *software* dalam proyek ini menunjukkan hasil yang stabil dan responsif tanpa adanya masalah signifikan seperti delay data atau kegagalan pengendalian perangkat. Hal ini menunjukkan bahwa proses integrasi antara sensor, mikrokontroler, komunikasi SPI, serta perangkat output telah dilakukan secara tepat dan efektif. Dengan demikian, sistem yang dikembangkan ini memiliki potensi untuk diaplikasikan sebagai solusi otomatisasi penyiraman pada lahan pertanian atau taman, yang dapat membantu meningkatkan efisiensi penggunaan air dan menjaga kesehatan tanaman.

REFERENCES

- [1]Digilab, “Modul 8 SSF : SPI & I2C,” Google Docs, 2020.
<https://docs.google.com/document/d/1CsIbwLVUrsKjZ3YhyF0J-gsGWNu1RCQu7JTYMCx3cFY/edit?tab=t.0> (accessed May 11, 2025).
- [2]Digilab, “Modul 9 SSF : Sensor Interfacing,” Google Docs, 2019.
<https://docs.google.com/document/d/14D8bETDw8x-BbeWWfg2QrjEE1WJA17kAZVDu79iCzCs/edit?tab=t.0> (accessed May 11, 2025).
- [3]Digilab, “Modul 7 SSF: Interrupt,” Google Docs, 2019.
https://docs.google.com/document/d/1VW7j3k_scKyOlzo72scSMFU58EgT-TLoiMOshQ48TUA/edit?tab=t.0 (accessed May 11, 2025).
- [4]microchip, “AVR ® Instruction Set Manual AVR ® Instruction Set Manual,” 2021.
Accessed: May 11, 2025. [Online]. Available:
<https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-InstructionSet-Manual-DS40002198.pdf>
- [5]Digilab, “Modul 6 SSF: Timer,” Google Docs, 2019.
https://docs.google.com/document/d/1eYX23D9J5vi9YKXsKkMGc2uem-RJoQlTHA_5TkcDfn4/edit?tab=t.0 (accessed May 11, 2025).
- [6]Digilab, “Modul 4 SSF : Serial Port,” Google Docs, 2019.
https://docs.google.com/document/d/1rRWvBgL3Nsb_h10131A-1kiGQkrGGNLedYoeVIGR9zg/edit?tab=t.0 (accessed May 11, 2025).
- [7]Digilab, “Modul 2 SSF: Introduction to Assembly & I/O Programming,” Google Docs, 2019.
https://docs.google.com/document/d/1s84Y1xrGyJwWXQJgdbQL6bTaFFZtiOsA4_3YGouP1CY/edit?tab=t.0 (accessed May 11, 2025).

